

Phantom Events: 揭示区块链中日志伪造问题

Yixuan Liu, Yuxin Dong, Ye Liu, Xiapu Luo, IEEE 高级会员, Yi Li, IEEE 会员

Yixuan Liu 和 Yi Li 供职于新加坡南洋理工大学计算与数据科学学院。

Yuxin Dong 供职于中国北京大学软件与微电子学院。

Ye Liu 供职于新加坡管理大学计算机与信息系统学院。

Xiapu Luo 供职于中国香港理工大学计算学系。

中文译文

摘要

随着区块链技术的快速发展，交易日志（transaction logs）在多种应用中发挥着核心作用，包括去中心化交易所（DEX）、钱包、跨链桥以及其他第三方服务。然而，这些日志，尤其是基于智能合约事件的日志，极易受到操纵和伪造，从而在整个生态系统中造成重大安全风险。为解决这一问题，本文首次对基于 EVM 的区块链中的交易日志伪造现象进行了深入的安全分析，并将这一现象称为幻影事件（Phantom Events）。我们系统地建模了五类攻击，并提出了一种用于检测智能合约中事件伪造漏洞的工具。评估结果表明，在识别潜在幻影事件方面，我们的方法优于现有工具。此外，我们已在多个去中心化应用中成功识别出五类攻击的真实世界实例。最后，我们呼吁社区开发者采取主动措施，以应对这些关键安全漏洞。

I. 引言

区块链是一种去中心化的数字账本技术，能够在多台计算机之间安全地记录和验证交易，从而确保数据完整性和透明性 [47]。比特币（Bitcoin, BTC）是区块链生态系统中最具代表性的代币，其市值在 2024 年 11 月达到了 18,000 亿美元，这体现了该技术的影响力和成功 [48]。

在区块链技术基础上，去中心化应用（Decentralized Applications, DApps）已成为一项关键创新 [42]。这些应用运行在部署于不同区块链网络上的智能合约之上，例如以太坊（Ethereum）、Polygon 和 Binance Smart Chain（BSC）。DApp 的扩展体现了应用开发向去中心化方向的转变，并提供了去中心化金融（Decentralized Finance, DeFi）和 GameFi 等多样化服务。该不断演进的生态系统由广泛的基础设施支撑，包括加密货币钱包和区块链浏览器，这些基础设施共同促进了区块链格局的活跃发展 [43]。

智能合约事件和交易日志对于跟踪区块链应用中的操作至关重要，并支撑跨链桥、钱包和 NFT 市场等场景中的功能。伪造这些日志可能破坏安全性，导致资产盗窃、欺诈以及互操作性问题，从而威胁用户信任。尽管既有研究已经考察了特定领域中的漏洞，例如跨链桥 [24]、[53]、[54]、NFT 所有权 [46]、代码不一致 [5] 以及代币行为 [59]，但目前仍缺乏针对多种应用场景下事件伪造问题的综合研究。本文通过提出幻影事件（Phantom Events）这一新型漏洞类别来弥补这一空白，该类别揭示了区块链上日志伪造的广泛风险。不同于以往聚焦于孤立场景的研究，本文系统总结了去中心化应用中的事件误用问题，并探索了用于识别幻影事件的不同检测方法。在既有研究中，XScope [24] 和 XGuard [54] 等检测工具工作在源代码层面，因此在只有字节码可用时检测能力受限。相反，SmartAxe [53] 和 TokenScope [59] 虽然分析字节码，但其仅基于字节码的方法会遗漏事件参数细节。Guidi 等人 [46] 的工作仅关注交易日志，由于上下文分析有限，误报率较高。我们的工具结合字节码、源代码（在可用时）和交易数据，能够对多种漏洞进行综合检测，同时降低单层分析方法所面临的误报。

幻影事件是区块链系统中的细微操纵行为，它们将正常的合约事件转变为未经授权操作的隐藏通道。这些事件削弱了对区块链交易的信任，使未经授权的活动能够绕过常规安全检查。幻影事件会模仿或轻微修改智能合约事件，从而误导事件监听器、用户界面和区块链分析工具。这类漏洞利用会破坏数据完整性，并可能造成重大的财务和声誉损失。例如，2024 年 2 月 9 日，攻击者通过伪造转账事件造成了 104 万美元的损失 [49]。

我们的研究聚焦于理解并分类不同类型的幻影事件。我们对近期区块链安全报告和文章进行了全面调研，以收集利用幻影事件相关漏洞的真实世界攻击。此外，我们审计了多个活跃智能合约，

并开发了一个用于处理区块链事件的安全模型（见第 III 节），该模型构成了分析和分类所收集攻击场景的基础（见第 IV 节）。进一步地，我们开发了一个名为 PEventCatcher 的工具，用于检测与幻影事件相关的漏洞（见第 V 节）。PEventCatcher 的设计面临若干挑战：（1）许多合约仅以字节码形式存在，缺乏事件语义，这使分析更加复杂；（2）幻影事件会模仿合法事件的格式，因而难以辨别；（3）需要理解合约的业务逻辑，才能判断是否存在异常。

利用我们的安全模型和检测工具，我们在不同区块链应用中识别出了大量漏洞和潜在风险（详见第 VI 节）。结果表明，幻影事件漏洞具有广泛性，并影响区块链生态系统中的多类应用。通过审计，我们发现了此前未被报告的不一致日志漏洞实例，在这些实例中，链上问题会导致其与链下记录之间出现差异。此外，我们的工具识别出了链上此前未被检测到的历史事件问题，凸显了该工具发现既往事件的有效性。在合约层级分析中，我们的工具优于现有方案，在检测幻影事件漏洞方面表现出更高的准确性。此外，我们在多个真实世界应用中识别出了安全问题，包括加密货币钱包、区块链浏览器和跨链桥，并已将这些发现报告给项目团队，以负责任披露方式申请漏洞赏金。最后，我们在第 VIII 节提出了缓解这些漏洞的策略。

简而言之，本文作出如下贡献：

- **新的攻击分类。** 我们是较早系统分析和分类区块链交易日志伪造相关攻击的研究之一。我们提出了一个能够刻画五类攻击的安全模型。我们还识别出了此前未被报告的不一致日志漏洞，在这些漏洞中，链上问题会导致其与链下记录之间存在差异。
- **高效检测。** 我们开发了一种检测机制，在识别幻影事件方面超过了现有工具，并成功发现了此前未被报告的攻击交易。
- **实践意义。** 我们在多种区块链应用中识别出了真实世界问题，包括区块链浏览器、加密货币钱包、GameFi、DeFi 和 NFT 市场。值得注意的是，我们在一个跨链中继器中发现了漏洞，而该中继器也被其他项目使用，其中包括一个市值超过 2.5 亿美元的项目。截至目前，我们已向项目团队和漏洞赏金平台报告了六个加密货币钱包问题、一个区块链浏览器问题以及一个跨链桥问题，其中五个已确认、一个已修复，并通过负责任披露共获得 600 美元赏金。

II. 背景

A. 基于 EVM 的区块链中的智能合约事件和交易日志

智能合约由 Solidity [30] 和 Vyper [31] 等高级语言编写，并被编译为字节码以在以太坊虚拟机（Ethereum Virtual Machine, EVM）[18] 上执行。基于 EVM 的区块链支持从简单转账到复杂 DApp [20] 的广泛功能。在这些区块链中，事件（events）和交易日志（transaction logs）支持智能合约与外部系统之间的通信。当智能合约发出一个事件时，该事件会被记录为交易日志，其中涉及如下关键组成部分。

事件。 以 Deposit 为例，事件可定义为一个元组 $\$E_{\{\text{Deposit}\}}(P_1, P_2, \dots, P_n)\$$ ，其中参数 $\$P_i\$$ 可以是索引参数（indexed，存储在主题 topics 中），也可以是非索引参数（non-indexed，存储在数据 data 中）。例如，在事件 `Deposit(address indexed account, uint256 amount, uint256 timestamp);` 中，`account` 是一个索引参数，这意味着它会被包含在事件的主题中，从而支持在事件日志中进行更高效的搜索和过滤。`amount` 和 `timestamp` 参数不是索引参数，但它们仍然提供了有关该交易的重要信息。

监听器。 监听器是指通过 RPC 方法订阅日志以监控特定事件的外部系统，例如 DApp 或钱包。

交易。 交易 $TX_{\{hash\}}$ 表示区块链上的一次操作，例如调用智能合约中的函数 $f_{\{name\}}$ 。它包括发送方地址和接收方地址、转账金额，以及定义函数调用及其参数的输入数据。

发送者。 发送者 $TX_{\{sender\}}$ 是创建交易的用户地址。

发出者。 发出者 $S_{\{address\}}$ 是在满足特定条件后触发事件的智能合约。

交易日志。 日志 $L_{\{hash\}} = (Topics_0^i, \allowbreak\{ Data^i, \allowbreak\{ S_{\{address\}}^i \})$ 在智能合约执行期间发出事件时生成。 $Topics_0^i$ 表示第 i 个日志的事件签名， $Data^i$ 包含非索引参数， $S_{\{address\}}^i$ 是发出该事件的合约。这些日志被存储在区块链上，并被引用以进行事件监控。

B. 代币

在基于 EVM 的区块链中，代币以智能合约形式实现，并遵循 ERC-20 和 ERC-721 等标准；这些标准定义了接口和事件，以确保 DApp 之间的兼容性。例如，ERC-20 规定了 **Transfer** 和 **Approval** 等事件，用于跟踪代币转账和授权；而 ERC-721 也定义了类似事件，但其适用于每个代币都唯一的 NFT。这些标准化事件使钱包和交易所等外部系统能够监控代币活动，并实时更新用户余额或所有权记录，从而实现准确且高效的跟踪。

图 1：跨链桥的基本架构。

C. 跨链桥

一些 DApp 依赖链下模块来监控智能合约发出的交易和事件，以便实时获知链上活动和状态变化。例如，跨链桥利用这些事件来触发链下流程、更新用户界面或执行其他操作。

在跨链桥场景中，如图 1 所示，源链（source chain）上的智能合约作为发出者 $S_{\{source\}}$ ，在用户存入资产时发出诸如 **Deposit** 的特定事件。目标链（destination chain）上的智能合约则作为参与者，由链下中继器调用，以处理已存入的资产并完成转账操作。

当用户在源链的桥合约（bridge contract）中存入代币时，存款事件 $E_{\{Deposit\}}$ 被发出并记录为交易日志 $L_{\{deposit_{\{tx\}}\}}$ 。链下中继器（off-chain relayer）作为监听器，监控这些日志以检测 **Deposit** 事件，处理事件信息，并与目标链上的智能合约协调以完成资产转移。

III. 威胁模型与动机示例

本节提出一个用于系统分析事件相关攻击的威胁模型，并通过动机示例说明幻影事件的影响。

A. 事件交互中的信任边界

在复杂的区块链交互环境中，建立清晰的信任边界对于全面理解与交易和事件工作流相关的安全风险及漏洞至关重要。图 2 所示的交互模型定义了三个主要信任边界。

图 2：区块链交互中的信任边界。

信任边界-区块链 ($\mathcal{M}_B$)。核心信任边界是区块链，它作为系统的骨干，用于存储交易和智能合约事件。在该边界内部存在交易存储 (Transaction Storage, M_{TS})，其可分为幻影事件和真实事件 (authentic events)。幻影事件是人为创建或操纵的事件，可能触发非预期操作；真实事件则是区块链交易的预期输出。智能合约区域 (Smart Contract, M_{SC}) 可分为真实合约 (authentic contracts) 和伪造合约 (forged contracts)，这表明合约既可能按预期运行，也可能执行恶意行为。其他区块链模块 (Other Blockchain Modules, M_{OM}) 包括共识机制、节点通信协议以及支持区块链功能的其他组件。

信任边界-DApp (M_D)。第二个边界包含作为用户与区块链之间接口的 DApp。DApp 监听来自区块链的事件并作出相应响应，通常会根据这些事件触发交易或更新。

信任边界-其他 (M_O)。第三个边界包括更广泛的生态系统，例如链下服务、外部 API 和钱包。这些组件同样依赖区块链事件的完整性来实现正确功能。

B. 威胁模型

在我们的威胁模型中，攻击者试图通过发出伪造事件来破坏 DApp 及更广泛的生态系统基础设施，并且还可能利用这些攻击通过社会工程策略误导用户。普通用户通过调用真实合约发出合法事件，而攻击者可以通过多种方式发出伪造事件：调用伪造合约、调用真实合约，或者调用一个伪造合约，再由该伪造合约调用真实合约以发出伪造事件。攻击者的能力被限定为直接进行合约调用或与 DApp 交互。

C. 动机示例

由于交易日志伪造，现实世界中已经发生了大量区块链攻击。例如，Qubit Bridge 和 pNetwork Bridge 攻击 [28]、[44] 分别造成了 8,000 万美元和 430 万美元的损失，其原因均在于幻影事件漏洞被利用。

图 3: pNetwork Hack 的攻击序列。

在 pNetwork 攻击中，事件处理机制存在缺陷：如果恶意合约和合法合约在同一笔交易内都被调用并且都发出事件，该机制会错误地将所有已发出的事件都视为合法合约事件。因此，如攻击序列（图 3）所示，攻击者首先在源链上部署了恶意合约 $S_{\{malicious\}}$ ，该合约通过函数调用来调用合法合约 $S_{\{legitimate\}}$ 。在合法合约 $S_{\{legitimate\}}$ 执行其函数 $f_{\{deposit\}}$ 并发出真实事件 $E_{\{Deposit\}}$ 后，恶意合约 $S_{\{malicious\}}$ 同时发出一个带有不真实金额的伪造事件 $E_{\{Deposit\}}$ 。由于真实事件和伪造事件都被记录在同一交易 $TX_{\{deposit\}}$ 下，中继器便将伪造事件当作合法事件处理。这使得攻击者能够利用系统无法区分这两个事件的缺陷，将未经授权的资金转移到目标链。

在 Qubit Bridge 攻击中，攻击者利用了受害合约 `deposit` 函数中的缺陷，如图 4 所示。中继器通过读取源链上发出的事件中的代币地址和金额，在目标链上转移资产。然而，攻击者利用 `deposit` 函数发出了本应只能由 `depositETH` 函数发出的事件，使其能够在源链上并未实际存入任何 ETH 的情况下触发目标链上的 ETH 转账，从而绕过中继器检查并从攻击中获利。

PEventCatcher 的思路。受这些真实世界示例启发，我们针对不同类型的漏洞采用定制化检测方法。幻影事件可能源于智能合约逻辑问题，也可能源于链下处理器缺陷。为此，我们设计了一种智能合约层级方法，用于识别可能导致幻影事件的漏洞。此外，我们还为交易数据开发了一种

链上方法，用于检查交易中可能表明攻击的模式或异常。这些方法相互补充，并形成了一个综合检测框架，能够同时处理基于合约和基于交易的幻影事件来源。

IV. 攻击分类与分析

本节提出由幻影事件引发的攻击分类，并为每种攻击提供详细解释以及检测规则。

表 1：攻击向量与相关模块之间的映射关系。

层级	攻击	描述	相关模块
链上	事件仿冒 (event counterfeiting)	使用已有合约发出幻影事件	M_TS, M_O
链上	不一致日志 (inconsistent logging)	数据库与区块链事件发出之间存在不一致	M_SC, M_TS, M_D
链下	合约仿冒 (contract imitation)	部署恶意合约以发出幻影事件	M_SC, M_TS, M_O
链下	转账事件欺骗 (transfer event spoofing)	结合社会工程攻击发出幻影事件	M_SC, M_TS, M_O
链下	事件处理错误 (event handling error)	由于错误的事件处理导致错误显示、插入或存储	M_D, M_O

我们依托行业报告、学术文献和真实世界智能合约审计，构建了一个相对较新且研究不足的幻影事件攻击分类体系。我们建模了五种不同攻击场景，并根据其来源将其分为两类：（1）链上（智能合约）漏洞；（2）链下弱点。这些攻击总结于攻击分类表（表 1）中。

这些攻击的详细演示可见补充材料。脚注：<https://github.com/PhantomEvent/Event-attack-demo>

A. 攻击向量与检测规则

本节介绍五种与幻影事件相关的攻击，并提出相应检测规则。这些攻击的示例可见附录 A。

1. 事件仿冒

图 4：事件仿冒攻击的概念验证，其中 `deposit` 和 `depositETH` 可能发出相同事件。

```
function depositETH(uint destinationChainId) external payable {
    require(msg.value > 0, "Deposit amount must be greater than 0");
    ethBalances[msg.sender] += msg.value;
    emit Deposit(msg.sender, msg.value, address(0), destinationChainId);
}

function deposit(address token, uint amount, uint destinationChainId) external {
    require(amount > 0, "Deposit amount must be greater than 0");
    safeTransfer(token, address(this), amount);
    tokenBalances[msg.sender][token] += amount;
    emit Deposit(msg.sender, amount, token, destinationChainId);
}
```

```
function safeTransfer(address token, address to, uint value) internal {
    (bool success, bytes memory data) = token.call(abi.encodeWithSelector(0xa9059cbb, to, value));
    require(success && (data.length == 0 || abi.decode(data, (bool))), "!safeTransfer");
}
```

该攻击利用合法合约在多个函数或执行路径中发出同一事件这一事实 (M_TS)。在图 4 的代码示例中, `depositETH` 和 `deposit` 两个函数都会发出 `Deposit` 事件, 其中第三个参数表示代币类型。`depositETH` 使用 `address(0)` 表示 ETH, 而 `deposit` 函数使用代币地址表示 ERC-20 代币。然而, `deposit` 函数也可以发出一个第三个参数为 `address(0)` 的事件, 从而模仿 ETH 存款。

攻击者可以通过调用 `deposit` 函数并将第三个参数伪造为 `address(0)` 来利用这一点, 而无需实际转入任何 ETH。这会创建一个看似真实 ETH 存款的幻影 `Deposit` 事件。依赖事件日志进行交易验证的链下验证系统, 可能会错误地将该事件视为合法 ETH 存款。由于事件验证过程存在缺陷, 攻击者由此可在目标链上欺诈性地申领大额资金。

检测方法。 当不同函数或执行路径以相同参数发出同一事件, 但各函数施加的约束不同, 此类漏洞便会出现。链下系统通常会在未验证参数约束的情况下将这些事件视为真实事件。如果某个函数发出一个本应由另一函数约束或验证的值, 则可将其视为潜在漏洞。检测规则检查不同执行路径上是否发出了相同参数值, 即使这些路径本应执行不同约束; 若存在这种情况, 则将此类事件标记为潜在漏洞。

形式化检测规则如下:

```
$$
\text{EC} = \left\{
\begin{array}{l}
e \in E \mid \\
\begin{array}{l}
& V(f, e) \cap V(g, e) \neq \emptyset \text{ and } \\
& \exists f, g \in F(e)
\end{array}
\end{array}
\right\}
$$
```

- E 是已发出事件的集合, $e \in E$ 表示单个事件。
- F 是合约中可发出事件的所有函数路径集合, $f, g \in F$ 是发出同一事件 e 的特定函数路径。
- $V(f, e)$ 和 $V(g, e)$ 表示事件 e 的参数在函数路径 f 和 g 上可能取到的值范围。

2. 不一致日志

图 5: 不一致日志攻击的概念验证。

```
function requestWithdraw(uint256 _type, uint256 _amount) external {
    require(WITHDRAW_ALLOWED, "TroyEmpire: Withdrawal is disabled for now");
}
```

```
emit WithdrawalRequested(_msgSender(), _type, _amount);
}
```

许多 DApp 采用混合日志模型，同时使用区块链和传统数据库来记录关键操作。例如，DeFi 平台记录存款和取款等金融交易，而 GameFi 平台跟踪用户操作。我们的分析显示，许多项目不仅依赖数据库记录，还会在区块链上发出日志。

具体而言，设计不佳的智能合约可能允许攻击者发出伪造事件，从而导致区块链日志与数据库日志之间不一致。例如，某个金融平台可能同时使用区块链和链下数据库记录取款请求。如果智能合约缺乏适当的访问控制，攻击者可以在区块链上任意生成取款事件，从而与数据库记录产生不匹配，如图 5 所示。

检测方法。 当智能合约在未对存储数据实施适当访问控制或验证的情况下发出事件时，此类漏洞便会出现。具体来说，一些合约缺少必要约束，允许任意用户在未针对合约状态验证事件参数的情况下触发取款等关键事件。为检测该问题，我们从两个关键方面分析合约函数：（1）是否存在访问控制机制或事件参数验证机制（例如 `require` 语句），以限制谁可以发出事件；（2）函数在发出事件时是否与存储交互，即是否读取或写入存储变量，以确保参数是针对先前存储的值进行验证的。如果某个函数在缺乏充分访问控制的情况下发出事件，或未与存储交互，则将其标记为潜在漏洞。

形式化检测规则如下：

\$\$

$\text{IL} = \left\{ \right.$

$f \in F \mid$

$\begin{aligned} & \text{Constraint}(f) = \emptyset \text{ or } \backslash \end{aligned}$

$\& (S_{\text{read}}(f) = \emptyset \text{ and } S_{\text{write}}(f) = \emptyset)$

$\end{aligned} \right.$

$\right\}$

\$\$

•

$\text{Constraint}(f)$: 函数 f 的访问控制机制或事件参数约束。如果 $\text{Constraint}(f) = \emptyset$ ，则不存在访问控制或约束。

•

$S_{\text{read}}(f)$ 和 $S_{\text{write}}(f)$: 函数 f 读取和写入的存储变量。

3. 合约仿冒

该攻击涉及部署恶意合约以利用或模仿已有合约（M_SC）。我们根据其特征将该攻击分为两个子类型。第一种子类型称为混合事件攻击（Blended Event Attack），发生在恶意合约与合法合约交互，并导致两个合约的事件被记录在同一交易中时。这种事件混合使得合法活动与欺诈活动之间难以区分。第二种子类型称为仿冒合约攻击（Mimicry Contract Attack），攻击者部署一个模仿合法合约行为的伪造合约，从而能够操纵事件日志和交易细节。

真实合约通常记录来自自身函数的日志。然而，大多数合约允许外部调用，恶意合约因而能够调用其函数，导致来自两个合约的日志被记录在同一交易中（M_TS）。该场景被称为混合事件攻击，在未验证日志发出者时会引入安全风险。

仿冒合约攻击利用了透明化实践，因为许多 DApp 团队会公开合约源代码以增强信任。这使攻击者能够修改并重新部署代码，创建模仿合法合约的恶意合约，并发出可被自由操纵的事件日志（M_TS）。对于 ERC-20 代币或 NFT，这种操纵使攻击者能够伪造交易记录，例如铸造或转移代币或 NFT；其中还可以伪造发送者，使 NFT 看起来仿佛由知名艺术家发行。

检测方法。 为检测此类攻击，我们分析交易日志。具体而言，我们检查同一事件签名

（Topics_0）是否在同一交易中多次出现，但由不同合约发出。如果该事件签名在同一交易中既由真实合约发出，也由伪造合约发出，则将其标记为潜在攻击交易。该检测规则有助于识别事件签名匹配但发出者不同的情况，这表明可能存在混合事件攻击。

\$\$

$$\begin{aligned} \text{L}_{\text{hash}} \mid \\ \begin{aligned} & \text{Topics}_0^i = \text{Topics}_0^j \text{ and } \\ & \text{S}_{\text{address}}^i = \text{S}_{\text{auth}} \text{ and } \\ & \text{S}_{\text{address}}^j = \text{S}_{\text{forge}} \end{aligned} \end{aligned}$$

right}

\$\$

4. 转账事件欺骗

该攻击是一种社会工程攻击 [21]。一个常见示例是零转账骗局（Zero Transfer Scam），其中攻击者创建一个模仿合法代币转账的幻影事件。在用户发送代币后，攻击者生成一个伪造事件，使其看起来像是用户将代币发送到了一个名称相似的接收地址，而该地址实际由攻击者控制。该伪造事件会被钱包和区块链浏览器记录，从而误导受害者向攻击者账户转移额外资金。据报道，该骗局已造成至少 2,736 万美元损失，并影响 28,414 名受害者 [51]。

第二种变体称为空投骗局（Airdrop Scam）[6]，它利用了 Web3 营销中常见的代币空投策略。攻击者通过使用真实 ERC-20 或 ERC-721 合约冒充发送者地址来伪造幻影事件，并且通常会选择容易引起注意的地址，例如 0x8888...8888。这些幻影空投诱使受害者与相关代币交互。攻击者可能设置蜜罐合约，使用户可以买入代币但无法卖出；也可能使用幻影事件日志推广恶意链接，例如使用 ENS 名称欺骗受害者。在更严重的情形下，攻击者可能实施 rug pull，抽走流动性并让受害者持有毫无价值的代币 [29]。

检测方法。 攻击者利用了许多第三方服务（例如钱包和区块链浏览器）盲目信任智能合约所发事件，而不验证其真实性这一事实。通过利用这种缺乏验证的问题，攻击者可以生成模仿合法转账的幻影事件，误导用户向攻击者地址转移资金。为检测该问题，我们可以分析交易日志，确保转账事件确由正确发送者发起，或者验证发送者是否已授权发起该交易的地址。

```

$$
\text{TS} = \left\{
L_{\text{hash}} \mid
\begin{aligned}
& \text{TX}_{\text{sender}} \neq \text{TokenSender} \text{ and } \\
& \text{Approve}(\text{TokenSender}, \text{TX}_{\text{sender}}) = \text{false}
\end{aligned}
\right\}
$$

```

- `TokenSender` 表示事件中的代币发送者。
- `Approve(TokenSender, TX_{sender})` 表示代币发送者是否已授权交易发起者转移其代币。

5. 事件处理错误

该攻击利用区块链浏览器、钱包和监控工具在处理和解释交易数据时存在的漏洞。这些应用依赖链上数据向用户提供实时信息。当幻影事件被错误地视为合法事件时，可能导致钱包余额不准确、交易历史虚假或资产所有权记录错误，从而误导用户，并可能影响其决策。

例如，在合约仿冒攻击场景中，即使幻影事件并非专门针对区块链浏览器，它们仍可能导致浏览器将这些事件记录为有效的 ERC-20 或 NFT 转账，尽管实际并未发生任何转账。这种误解使攻击者能够在缺乏底层交易的情况下制造转账表象。

攻击者还可以进一步利用该问题，在事件数据中嵌入恶意载荷，以发起跨站脚本（Cross-Site Scripting, XSS）或 SQL 注入（SQL injection, SQLi）等攻击。如果缺乏适当清洗，此类载荷可能导致 DApp 界面上的未授权操作、数据盗窃或数据库被破坏。

检测方法。 该攻击利用了区块链浏览器、钱包和监控工具对链上数据的隐式信任。有效检测需要针对每个链下应用的需求设计适应性方法。浏览器应基于实际代币转账验证事件，而钱包需要确认发送者是否被允许进行转账。通过依据这些独特的验证标准定制检测规则，应用可以缓解幻影事件和嵌入式恶意载荷带来的风险。

V. 漏洞检测

图 6: PEventCatcher 的架构。

本节聚焦于检测第 IV 节所讨论攻击向量相关的漏洞。我们介绍一种混合方法，该方法结合静态分析、符号执行和链上数据监控，以确保全面的漏洞检测。

A. PEventCatcher 框架设计

如图 6 所示，我们的检测方法采用多层级策略，在交易层级、字节码层级和源代码层级进行漏洞检测。

在交易层级，由于真实事件和发出者在不同项目之间存在差异，我们手工研究了不同项目的文档，以建立适当规则。这些规则被应用于历史和实时链上交易数据，用于通过识别幻影事件与真实事件混合的实例来检测合约仿冒。

对于智能合约，我们的分析跨越字节码和源代码两个层级。首先，我们使用 Gigahorse [52] 将智能合约字节码反编译为三地址形式的中间表示（intermediate representation, IR）。基于该 IR，我们构建以基本块为基础的跨合约控制流图（inter-contract control flow graph, ICFG）。该图支持后向污点分析，我们利用该分析追踪事件相关路径，并检测与事件仿冒和不一致日志相关的漏洞。

如果源代码可用，我们进一步使用 Slither 执行源代码层级分析。这通过追踪事件相关调用路径并提取用于符号执行的约束，为事件仿冒提供二次确认，使我们能够更全面地验证参数值。

由于某些攻击具有独特性质，事件处理错误缺乏特定检测方法，因为其高度依赖链下系统验证。类似地，转账事件欺骗主要是一种社会工程攻击，重点在于代币行为分析。因此，我们的工具主要针对其余三类攻击进行检测：事件仿冒、不一致日志和合约仿冒；这些攻击具有明确模式和合约层级漏洞，能够被有效识别和处理。

B. 交易层级检测

在交易层级，链下监控被用于分析链上交易数据并检测潜在合约仿冒攻击。通过基于智能合约预期行为建立领域特定规则，我们系统地验证交易是否遵循正确逻辑，并识别任何可能表明幻影事件或未授权交互的恶意活动迹象。

该分析包括解析交易数据，并应用一组规则来验证事件的关键方面，包括：（1）确保发出的事件具有正确签名；（2）验证事件发出者是否与预期合约地址匹配；（3）检查函数执行路径是否符合预期逻辑；（4）确认事件参数值是否与预期一致。

通过执行这些规则，我们可以检测可能表明合约仿冒攻击的异常。如果检测到攻击交易，则会进行进一步分析以确认其影响，例如是否导致未经授权资金转移或状态变化。

表 2: pNetwork 攻击的交易日志。

事件	参数	发出者	参数值
Burned	operator, from, amount, data, operatorData	pBTC	(Attacker, Attacker, 0, , _)
Transfer	from, to, amount	pBTC	(Attacker, address(0), 0)
Redeem	redeemer, value, underlyingAssetRecipient, userData	pBTC	(Attacker, 0, Attacker_Bitcoin_address, _)
Redeem	redeemer, value, underlyingAssetRecipient, userData	Malicious Contract	(Attacker, 274[...]144, Attacker_Bitcoin_address, _)

例如，在 pNetwork 攻击中（见表 2），源链上一个恶意合约以非预期合约地址发出了 Redeem 事件。应用这些交易层级规则使我们能够检测该攻击。随后对目标链交易的监控确认了该攻击通过未经授权资金转移取得成功。

C. 智能合约层级检测

算法 1：通过后向污点分析检测不一致日志和事件仿冒。

输入：IR，通过反编译得到的智能合约字节码中间表示。

输出：\$E_v\$，漏洞事件集合。

```

1 E_v = \emptyset.
2 construct ICFG out of IR.
3 extract LogOps, i.e., event log operations, from IR.
  # Each operation contains a event signature and a set of logging data variables
4 Vars = \emptyset,
5 for logOp in LogOps do
6   Vars <- Vars union vars(logOp)
  # Record all logging data variables
7   Slices <- BackwardSlicing(logOp, ICFG)
  # Each slice is a reverse execution flow path from logOp to function entry point
8   extract Paths, i.e., inter-functional paths based on Slices and ICFG
9   taintedPaths <- \emptyset
10  for p in Paths do
11    source_op <- TaintAnalysis(p, Vars)
12    if source_op != null then
13      if sstore(p[:source_op]) = \emptyset then
14        E_v <- E_v union event(logOp)
        # No storage operations
15      end if
16      taintedPaths <- taintedPaths union {event(logOp)}
17    else
18      if HasExternalCall(p) and not HasJumpi(p) then
19        E_v <- E_v union event(logOp)
        # No constraints
20      end if
21    end if
22  end for
23  if ||taintedPaths|| > 1 then
24    E_v <- E_v union event(logOp)
    # A event logging action has multiple tainted paths
25  end if
26 end for
27 return E_v

Procedure TaintAnalysis(path, taintVars)
28 entry = path[-1]
  # Function entry point is the last item of this reverse execution path
29 for instr in path do
30   if vars(instr) intersect taintVars != \emptyset then
31     taintVars <- taintVars union vars(instr)
    # Taint propagation
32   end if
33 end for
34 if vars(entry) intersect taintVars != \emptyset then
35   return entry
36 else
37   return null
38 end if
EndProcedure

```

在智能合约层级，我们通过一种由两部分组成的多层分析方法，检测与事件仿冒和不一致日志直接相关的漏洞：（1）在字节码层级通过跨过程后向污点跟踪识别潜在幻影事件；（2）在源代码层级通过验证事件参数约束来确认漏洞。这种双方法设计使我们能够分析合约的不同表示形式，从而识别在单一抽象层级上可能不可见的漏洞。

1. 通过跨过程后向污点跟踪识别幻影事件

在字节码层级，如算法 1 所述，与事件仿冒和不一致日志相关的漏洞检测通过对智能合约字节码的中间表示（IR）执行后向污点分析完成。该算法通过分析数据从事件日志操作到其来源的流动，并检查表明漏洞存在的特定条件，识别漏洞事件 `E_v`。

算法首先初始化一个空集合 `E_v`，用于存储漏洞事件（第 1 行）。随后从 IR 构建跨过程控制流图（ICFG）（第 2 行），并从 IR 中提取所有事件日志操作（LogOps）（第 3 行）。每个日志操作由一个事件签名和一组事件数据变量组成。算法随后初始化空集合 `Vars`，用于记录事件变量（第 4 行）。

ICFG 的构建涉及通过将字节码中的哈希标识符与签名数据库中的条目匹配，恢复函数级信息，例如函数名和事件签名。随后使用表示每个函数内可能执行路径的基本块构建控制流图（control flow graph, CFG）。这些基本块根据控制流指令和跳转目标通过有向边连接，形成 CFG。为了捕获跨函数交互，算法识别跨过程调用，并在调用者函数与被调用者函数之间添加边，从而将 CFG 扩展为 ICFG。

对于 LogOps 中的每个日志操作，算法使用该日志操作涉及的变量更新 `Vars`（第 6 行）。随后在 ICFG 上执行 BackwardSlicing 分析，以提取表示从日志操作到函数入口点的反向执行流的切片（第 7 行）。这些切片随后被用于推导跨函数路径（Paths），以进行进一步分析（第 8 行）。

对于 Paths 中的每条路径 `p`，算法使用 TaintAnalysis 函数执行污点分析（第 9 行）。该函数评估污点数据是否从日志操作传播到路径入口点。从日志操作开始，污点变量通过反向执行路径中的指令传播进行跟踪。具体而言，如果某条指令与污点变量交互，则其相关变量会被加入污点变量集合。该过程持续到到达路径入口点，该入口点作为污点来源。如果在路径入口处发现污点变量，函数返回该入口点作为 `source_{op}`；否则返回 `null`。

如果识别到污点数据来源（`source_{op}`），算法检查通向该来源的路径片段中是否存在存储操作（`sstore`）（第 11 行）。此外，算法验证这些 `sstore` 操作涉及的变量是否受约束，例如是否存在类似 `jumpi` 的条件跳转。如果未发现 `sstore` 操作，或 `sstore` 操作缺乏对其变量的约束，则将对事件加入 `E_v`，作为潜在易受不一致日志影响的事件（第 12 行）。该事件还会被记录到单独的污点路径集合中，以便进一步评估（第 13 行）。

如果未识别到污点数据来源，算法检查路径是否包含没有相应约束的外部调用（`call`），例如是否缺少类似 `jumpi` 的条件分支（第 15 行）。若满足这些条件，则将该事件加入 `E_v`，视为易受事件仿冒影响（第 16 行）。

在分析完某个日志操作的所有路径后，算法检查同一事件是否存在多条污点路径（第 18 行）。如果存在，则由于存在多条发出路径，该事件被标记为易受事件仿冒影响（第 19 行）。

通过分析污点数据流并识别缺失约束、未检查外部调用以及缺乏存储操作等条件，该算法能够有效检测漏洞。`E_v` 中被标记的事件易受事件仿冒或不一致日志影响：前者意味着同一事件可能从多条路径被误导性发出，后者意味着日志数据可能缺乏控制。

2. 源代码层级的事件参数约束验证

字节码层级分析存在局限，因为它只能揭示索引事件参数，而无法访问事件的完整语义。因此，字节码层级分析只能通过识别索引参数中的不一致来检测潜在不一致日志问题，无法处理非索引参数。为实现完整验证，需要进行源代码分析。在源代码层级，我们能够同时访问索引参数和非

索引参数，从而对潜在漏洞进行全面评估，包括事件仿冒和不一致日志；这些漏洞无法仅靠字节码层级分析被完全发现。

为应对这些挑战，我们依赖合约源代码的完整语义上下文，追踪事件相关调用路径，并应用符号执行来验证参数约束。这确保了事件在所有路径上以一致值发出，并使索引参数和非索引参数都与预期合约逻辑保持一致。此外，源代码层级的事件仿冒检测涉及分析多条执行路径，以验证事件是否在多个函数之间被不当使用，从而提供字节码分析无法达到的验证深度。

算法 2：通过符号执行发现不一致日志和事件仿冒。

输入：SC，智能合约源代码。

输出：\$E_v\$，幻影事件集合。

```

1 E_v = \emptyset
2 extract E, i.e., all events from SC.
3 for event in E do
4   Paths <- SearchPaths(event, SC)
5   # Get all paths reaching the event
6   for p in Paths do
7     constraints <- symbolicExec(p)
8     if CheckLoggingInconsistency(constraints) then
9       E_v <- E_v union {event}
10    end if
11  end for
12  for p_1, p_2 in Paths do
13    constraints_1 <- symbolicExec(p_1)
14    constraints_2 <- symbolicExec(p_2)
15    if SMT-Solve(constraints_1 and constraints_2) then
16      E_v <- E_v union {event}
17    end if
18  end for
19 return E_v

```

在源代码层级，如算法 2 所述，工具首先初始化一个空集合 E_v ，用于存储潜在漏洞事件。随后从智能合约源代码 SC 中提取所有事件 E。对于 E 中的每个事件，工具执行如下步骤。

首先，工具使用 SearchPaths 函数提取 SC 中所有通向该事件发出的执行路径 Paths。这涉及遍历合约调用图，以捕获从用户可调用函数到事件发出语句的所有跨过程路径。对于每条路径 $p \in \text{Paths}$ ，工具使用 symbolicExec 执行符号执行，以提取与事件参数相关的路径约束。随后，工具应用 CheckLoggingInconsistency 函数，通过比较所有参数（包括索引参数和非索引参数）的约束来验证任一路径是否存在日志不一致。该函数类似于字节码分析，它通过检查变量约束缺失、未检查外部调用或存储操作缺失来检测漏洞。如果检测到不一致，则将该事件加入 E_v ，作为潜在易受不一致日志影响的事件。

接下来，工具通过考虑 Paths 中所有具有同一事件的不同路径对 (p_1, p_2) 来执行事件仿冒检测。对于每一对路径，工具执行符号执行，以分别获得路径约束 constraints_1 和 constraints_2 。使用 SMT 求解器，工具检查合取式 $\text{constraints}_1 \wedge \text{constraints}_2$ 是否可满足。如果求解器找到一个解，则表明两条路径之间存在参数值重叠，使验证者难以区分来自不同路径的事件发出。在这种情况下，该事件被加入 E_v ，视为潜在易受事件仿冒影响。

最后，工具返回集合 E_v ，其中包含被标记为存在潜在漏洞的事件。例如，考虑图 4 中由 deposit 和 depositETH 函数发出的 Deposit 事件。工具在合约 AST 中识别这些函数，然后分析其各自执行路径以提取事件参数约束。对于 depositETH，约束为 $\text{msg.value} > 0$ ；对于 deposit，约束为 $\text{bool}(\text{token.call}(\text{signature})) \wedge (\text{amount} >$

0)。利用 SMT 求解器，工具评估这些约束以检查是否存在重叠参数值。如果存在交集，则表明可能存在事件仿冒，因为验证者可能无法区分来自这两个函数的事件发出，工具因而会将该事件标记为可伪造，以供进一步审查。

VI. 实现与实验

图 7：针对 9 个地址在不同平台上对 ERC-20 转账幻影事件的分析。

(a) BSC; (b) ETH; (c) Polygon。

我们的工具使用 Python 实现，支持智能合约的字节码层级和源代码层级分析，也支持交易层级监控。对于字节码层级分析，我们使用 Gigahorse 框架从编译后的字节码构建 ICFG。在源代码层级，我们的工具支持所有与 Slither [33] 兼容的 Solidity 版本，使我们能够提取 AST，并对事件发出和参数约束进行详细分析。我们使用 Z3 [7] 作为约束求解器，以处理符号执行并消除不可行路径。此外，该工具包含一个交易层级链下监控系统，用于解析链上数据，并应用自定义规则来检测异常和验证攻击交易是否成功。所有主要组件，包括字节码分析引擎、源代码符号执行器和交易监控系统，均由我们开发。实现基于 Python，并包含超过 5,000 行代码。

- **RQ1** 幻影事件在主要区块链平台上的普遍程度如何，PEventCatcher 在交易层级检测中的表现如何？我们旨在理解验证者通过合约地址和源函数验证事件来源的必要性。
- **RQ2** PEventCatcher 在从智能合约中检测潜在幻影事件方面的有效性如何？我们开发了 PEventCatcher，并构建了一个自定义基准来评估其有效性。
- **RQ3** 在真实世界中实施此类攻击的可行性如何？我们旨在验证所识别漏洞是否可在真实世界场景中被实施。

A. 幻影事件的普遍性 (RQ1)

TokenScope [59] 等工具已经对检测代币行为不一致作出贡献，Guoyi Ye 等人 [51] 也展示了转账事件中伪造用户地址的普遍性。我们旨在探索两个具体方面：幻影事件攻击中可疑地址的普遍性，以及此类攻击在跨链桥中是否常见。

1. 方法

为更好理解幻影事件在真实世界中的发生情况，我们使用区块链交易数据进行了两个独立实验。

在第一个实验中，我们聚焦于九个特定伪造地址，即从 `0x1111...1111` 到 `0x9999...9999`，因为这些地址极不可能在正常交易中生成或使用。我们检查了截至 2024 年 10 月 21 日在 Ethereum、BSC 和 Polygon 区块链上的交易，寻找源自这些地址的 ERC-20 代币移动。该方法有助于衡量幻影事件在不同区块链平台上的普遍性。

在第二个实验中，鉴于跨链桥极易受到幻影事件攻击，我们基于交易层级分析为跨链桥创建了检测规则。我们使用 XScope [24] 及其关联数据集，将我们的发现与其结果进行比较；XScope 是现有唯一用于检测桥攻击交易的工具。

2. 结果

在第一个实验中，我们聚焦九个特定地址，如图 7 所示，识别出 ERC-20 代币转账中存在大量幻影事件。我们在 Ethereum 上发现 18,099 个此类事件，在 BSC 上发现 1,245,125 个，在

Polygon 上发现 436,407 个。如果未被适当验证，这类事件可能导致安全漏洞利用。仅来自九个地址的显著数量便凸显了严格交易验证对于维持区块链操作安全性和可信度的关键必要性。

表 3：各工具检测到的攻击交易数量和成功攻击数量。

项目	XScope 攻击数	PEventCatcher 攻击交易数	PEventCatcher 成功攻击数
THORChain #1	9	9	9
THORChain #2	41	48	41
pNetwork	3	5	3
Qubit Bridge	20	20	20
meter.io	N/A	5	5
CENNZnet	N/A	1	1

在第二个实验中（结果见表 3），我们检测到了 XScope 报告的所有交易，并识别出额外的攻击交易。XScope 仅记录攻击已经发生的交易，即目标链上出现资金转移的交易。相比之下，我们能够在转账发生之前检测攻击，帮助项目团队主动识别攻击者。我们还检测到 meter.io [45] 上的攻击交易；XScope 在其攻击事件列表中提到了该项目，但并未进行分析。此外，我们在 CENNZnet 上发现了一笔攻击交易，造成 150 ETH 损失。该事件此前未被任何其他工具或安全公司披露或报告。

对 RQ1 的回答： 我们的分析表明，仅基于九个地址，Ethereum、BSC 和 Polygon 上就存在大量幻影事件。此外，与最先进工具相比，我们的检测规则展示了有效性。

B. 从代码检测幻影事件（RQ2）

1. 数据集

SmartAxe [53] 和 XGuard [54] 是目前已知的两个用于检测跨链漏洞的工具，其中 SmartAxe 聚焦于字节码层级分析，XGuard 提供源代码层级分析。

在对 SmartAxe 数据集的分析中，我们发现 20 个攻击中只有 8 个是由事件仿冒引起的，且漏洞函数路径对存在于桥合约内。我们重新标注了这 8 个攻击，并从 SmartAxe 分析的 129 个桥中增加了 29 个受事件仿冒影响的函数路径对。为补充该数据，我们从 BSC 上 452,666 个已验证合约 [38] 中使用 TF-IDF 算法选择了 500 个相似度较低的 BSC 智能合约，并对其进行人工标注。去重后，我们额外识别出 80 个受事件仿冒影响的函数对。

通过结合 SmartAxe 和 XGuard 数据集以及人工标注的函数对，我们组装了一个包含 117 个受事件仿冒影响的函数对的综合数据集，用于评估我们的检测工具。

为检测不一致日志，我们使用了一个包含 28 个人工审计的不一致日志参数案例的数据集，这些案例是在我们的审计过程中识别出的。

2. 结果

表 4: 事件仿冒与不一致日志的工具比较。

工具	事件仿冒 Precision	事件仿冒 Recall	事件仿冒 F1	不一致日志 Precision	不一致日志 Recall	不一致日志 F1
SmartAxe	40.50%	25.12%	31.01%	N/A	N/A	N/A
XGuard	87.75%	55.12%	67.70%	64.71%	91.75%	75.82%
PEventCatcher (Bytecode)	84.0%	100%	91.30%	62.50%	16.12%	39.31%
PEventCatcher (Source)	90.83%	86.51%	88.62%	90.30%	100%	94.90%

在我们的实验中，如表 4 所示，PEventCatcher 在检测事件仿冒和不一致日志漏洞方面均显著优于 SmartAxe 和 XGuard。对于事件仿冒，SmartAxe 的 F1 得分为 31.01%，Precision 为 40.50%，Recall 为 25.12%；XGuard 的 F1 得分为 67.70%，Precision 为 87.75%，Recall 为 55.12%。相比之下，PEventCatcher 表现更优，在字节码层级达到 91.30% 的 F1 得分，在源代码层级达到 88.62% 的 F1 得分。对于不一致日志检测，PEventCatcher 也优于 XGuard，在实现完美 Recall 的同时取得 94.9% 的 F1 得分，而 XGuard 的 F1 得分为 75.82%。

PEventCatcher 的字节码实现和源代码实现之间的性能指标差异，体现了各方法的特定局限。在事件仿冒检测中，PEventCatcher 的字节码层级分析缺乏约束验证，因此 Precision 较低，因为它可能会标记缺乏充分伪造证据的事件。然而，在源代码层级，PEventCatcher 会执行全面的约束检查，从而显著提高 Precision。对于不一致日志，PEventCatcher 的字节码分析仅限于索引参数，这降低了 Recall，因为它无法识别与非索引参数相关的问题。在可访问源代码时，PEventCatcher 能够同时分析索引参数和非索引参数，从而获得更高 Recall。相比之下，SmartAxe 和 XGuard 存在固有设计局限；SmartAxe 仅检测两个函数是否发出同一事件，而不考虑调用路径中的约束验证；XGuard 则聚焦单个函数内的约束，缺乏函数间分析。通过同时分析函数交互和调用路径，PEventCatcher 检测到了其他工具遗漏的漏洞，并在事件仿冒和不一致日志检测中取得了更优结果。

3. 局限性

我们评估中的误报和漏报揭示了该方法的两个主要局限。第一，在处理不完整合约代码时，例如某些函数中的接口调用，我们计算得到的约束可能并不总是精确，从而导致约束较弱并引发误报。第二，一些已由区块链浏览器验证的智能合约在使用我们的工具进行分析时因 Solidity 编译 bug 而无法编译 [39]，从而出现编译错误。解决这些局限将是我们未来工作的重点。

对 RQ2 的回答： PEventCatcher 已被证明在检测幻影事件漏洞方面高度有效，并在 Precision、Recall 和 F1 得分上超过当前最先进工具。

C. 真实世界攻击可行性 (RQ3)

1. 方法

我们将工具应用于多种区块链平台进行测试，旨在识别智能合约中与幻影事件攻击相关的漏洞。除链上测试外，我们还审计了多种链下应用，包括区块链浏览器、NFT 市场和加密货币钱包。这些审计聚焦于评估安全协议并识别可能被幻影事件利用的潜在漏洞，从而确保对链上和链下组件进行综合分析。

所有识别出的幻影事件漏洞均在受控本地环境中通过模拟场景进行测试，以确保不会对真实世界区块链平台造成中断或损害。

2. 结果

在实验中，我们识别出一笔转账事件欺骗交易，其中一个虚假转账看似将代币从地址 `0x8888...8888` 转移到了受害者的钱包。尽管实际并未发生代币转账，但至少五个主要区块链浏览器，即 BscScan、OKLink、Bitquery、Tokenview 和 Bsctrace [16]、[11]、[1]、[2]、[12]，都错误地将这一欺骗事件识别为合法交易。这凸显了链下系统中的广泛漏洞：这些系统难以区分真实事件和幻影事件。

此外，我们在两个领先 NFT 市场 Opensea 和 Rarible 的测试网环境中成功复现了“sleepminting”攻击。尽管已有监控方案旨在检测此类攻击，但完全缓解这些漏洞仍然是持续性挑战。

另外，在安全审计期间，我们在六个加密货币钱包中发现了关键漏洞，并已向相应项目团队或通过漏洞赏金平台报告。其中四个漏洞已被确认，一个漏洞从项目团队获得了 600 美元赏金。

在对若干区块链桥进行审计时，我们识别出链下代码中的漏洞，使其暴露于仿冒合约攻击之下。在该攻击中，恶意合约发出伪造事件，而链下系统会错误地将其作为合法事件处理。我们发现的一个易受攻击系统被某公共区块链项目 fork 并部署；截至 2024 年 10 月 9 日，该项目市值超过 2.5 亿美元。这些漏洞已报告给相应项目团队以进行修复。

此外，我们在流行的区块链浏览器 Blockscout [60] 中识别出事件数据展示问题，该问题可能损害交易记录的准确性。该缺陷可能导致用户误解区块链数据，从而削弱对交易历史的信任。

在对多个活跃智能合约项目的审计中，我们还发现三个 DeFi 项目和一个 GameFi 项目易受不一致日志攻击影响；该漏洞由本文首次提出（见第 IV-A2 节）。在这组项目中，受影响项目的最高市值为 169,688 美元。

对 RQ3 的回答： 我们的研究表明，幻影事件攻击在真实世界区块链平台上是可行的，具体目标包括区块链浏览器、NFT 市场、DeFi、GameFi 和加密货币钱包。这强调了区块链生态系统中改进安全措施的需求。详细报告和确认案例可见补充材料。

D. 有效性威胁

内部有效性威胁主要源于人工数据标注过程中可能引入的不准确性。为提高数据标注准确性，我们采用三层方法：由两名作者独立标注数据，再由第三名作者审查其标注。该方法在三个不同层级进行彻底审查，以提升数据集分类的精确性。

为确保外部有效性，我们使实验所用数据集的类型和来源多样化。除 SmartAxe 和 XGuard 提供的以桥为中心的数据外，我们将数据集扩展到更广泛的场景。此外，我们避免使用高度相似的智能合约，以提高分析的多样性和有效性。

VII. 相关工作

A. 智能合约安全分析

近年来，大量技术被提出用于分析智能合约安全性。

静态分析。 Tikhomirov 等人 [34] 设计了 SmartCheck，该系统将 Solidity 源代码转换为 XML，并通过 XPath 模式检测 bug。Grech 等人 [32] 提出了名为 Gigahorse 的静态分析框架，该框架将基于栈的字节码转换为基于寄存器的中间表示。Feist 提出了一个名为 Slither [33] 的类似工具，其面向 Solidity 源代码。Lu 等人 [40] 设计了一个名为 NeuCheck 的智能合约安全分析工具，该工具基于语法树解析。Ghaleb 等人 [63] 提出了 eTainter，这是一种静态分析工具，通过对字节码应用污点跟踪来检测智能合约中的 gas 相关漏洞。

符号执行。 Luu 等人 [19] 提出了 Oyente，这是一个开创性的以太坊智能合约符号执行工具，用于检测 bug。Lin 等人 [4] 提出了 SolSEE，这是第一个面向 Solidity 智能合约的源代码层级符号执行引擎。Ma 等人 [35] 引入了 Pluto，通过重构跨合约 CFG 来检测安全 bug。Pasqua 等人 [26] 提出了一种基于 EVM 操作数符号执行的方法，用于精确 CFG 构建并改进漏洞检测。Ruario 等人 [61] 实现了 CRUSH，该工具利用符号执行和程序切片来检测此类合约组中的存储冲突。Gritti 等人 [62] 开发了 JACKAL，它基于控制流图（CFG）和函数调用图（function call graph, FCG）执行符号执行，以检测 confused contract 漏洞。此外，Mythril [27] 和 Manticore [36] 等行业方案已成为智能合约审计的标准工具。

我们的方法涉及开发一个结合多种技术的检测框架，包括字节码层级分析、源代码分析、交易层级监控和符号执行，但本文所处理的问题无法被任何现有漏洞模式捕获。

B. 智能合约事件

一般而言，日志消息能够增强程序理解并降低维护成本，但关于 Solidity 事件日志和安全性的研究仍然有限。Li 等人 [23] 开展了首个关于 Solidity 事件日志实践的实证研究，并开发了一个用于识别导致不必要 gas 使用事件的工具。Zhang 等人 [24] 设计了名为 XScope 的工具，用于发现跨链桥中的安全违规事件。Cernera 等人 [25] 通过扫描日志寻找 Transfer 事件，跟踪由内部交易创建的代币。此外，Guidi 等人 [46] 分析了 sleepminting 现象，并探索了使用 Forta 对可疑事件进行跟踪和告警。Zhu 等人 [5] 提出了一种名为 DocCon 的技术，用于检测 Solidity 代码与其对应文档之间的不一致。这些不一致包括文档中记载但代码中未出现的事件发出，或相反情况。TokenScope [59] 通过监控 ERC-20 事件，为检测代币行为不一致作出了贡献。尽管已有上述进展，大多数研究仅考虑了特定场景中的事件相关问题，尚未充分处理由幻影事件引发攻击的一般性。

VIII. 缓解策略

缓解由幻影事件导致的漏洞需要一种综合方法，同时处理智能合约开发、生态系统基础设施和攻击检测机制。从合约开发角度看，开发者应实现严格的验证机制，确保事件参数在发出前得到验证，并为函数实现访问控制机制。强制执行适当的状态转换至关重要，以防止已发出事件与实际合约状态之间出现不匹配。

在生态系统层级，区块链浏览器、钱包和 DApp 等链下系统必须采用更稳健的验证技术，以区分合法事件与幻影事件。事件发出者验证会将事件来源与合约地址进行交叉检查，有助于确保事件来自授权合约。此外，改进链下应用中的数据清洗流程，对于防止跨站脚本（XSS）和 SQL 注入（SQLi）等漏洞至关重要。跨链桥需要增强的跨链安全协议，以确保源链和目标链上的事件均得到验证，从而防止事件伪造和操纵。

在安全攻击检测方面，对链上交易和事件进行持续实时监控，对于检测并标记可疑活动至关重要，例如转账事件欺骗或合约仿冒。针对链上合约行为和链下事件处理分别定义详细检测规则，可以更全面地识别漏洞。此外，还应定期对智能合约和链下系统进行安全审计，以识别潜在弱点，尤其应关注事件发出逻辑、访问控制和交易验证。通过综合运用这些策略，可以显著降低幻影事件带来的风险，并提高区块链系统的安全性和可靠性。

IX. 结论

在本文中，我们对区块链系统中与幻影事件相关的漏洞进行了深入分析。我们的方法包括开发一个多层级检测框架，该框架集成字节码层级、源代码层级和交易层级分析，以处理事件仿冒、不一致日志和合约仿冒等问题。此外，我们为所提出的每种攻击向量识别了真实世界案例，凸显了在区块链生态系统中缓解这些威胁的关键必要性。

致谢

原文该部分为占位内容，未提供正式致谢文本。

参考文献

- [1] Bitquery, “Bitquery”, 2023. [Online]. Available: <https://explorer.bitquery.io/>. [Accessed: Dec. 15, 2023].
- [2] Tokenview, “Tokenview”, 2023. [Online]. Available: <https://bsc.tokenview.io/>. [Accessed: Dec. 15, 2023].
- [3] Chainlink, “Chainlink”, 2023. [Online]. Available: <https://chain.link/>. [Accessed: Dec. 15, 2023].
- [4] S.-W. Lin, P. Tolmach, Y. Liu, and Y. Li, “SolSEE: a source-level symbolic execution engine for Solidity”, in Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2022, pp. 1687–1691.
- [5] C. Zhu, Y. Liu, X. Wu, and Y. Li, “Identifying Solidity Smart Contract API Documentation Errors”, in Proc. 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13.
- [6] H. Kamarul, “‘Spoof’ Tokens on Ethereum”, 2022. [Online]. Available: <https://medium.com/etherscan-blog/spoof-tokens-on-ethereum-c2ad882d9cf6>. [Accessed: Dec. 15, 2023].
- [7] L. De Moura and N. Bjørner, “Z3: An efficient SMT solver”, in Proc. International Conference on Tools and Algorithms for the Construction and Analysis of Systems, 2008, pp. 337–340.
- [8] Halborn, “Thorchain Incident Analysis”, 2023. [Online]. Available: [https://github.com/HalbornSecurity/PublicReports/blob/master/Incident Reports/](https://github.com/HalbornSecurity/PublicReports/blob/master/Incident%20Reports/). [Accessed: Dec. 15, 2023].
- [9] PeckShield, “PeckShieldAlert on X”, 2023. [Online]. Available: <https://twitter.com/PeckShieldAlert/status/1551630862600527874?t=pAmAETpV2jpmLagU23QD9w&s=05>. [Accessed: Dec. 15, 2023].

- [illegible]

- [28] R. Behnke, “EXPLAINED: THE PNETWORK HACK (SEPTEMBER 2021)”, 2023. [Online]. Available: <https://www.halborn.com/blog/post/explained-the-pnetwork-hack-september-2021>. [Accessed: Dec. 15, 2023].
- [29] N. Semczuk, “What is a rug pull?”, 2023. [Online]. Available: <https://www.bankrate.com/investing/what-is-a-rug-pull/>. [Accessed: Dec. 15, 2023].
- [30] C. Dannen, *Introducing Ethereum and Solidity*, vol. 1, Springer, 2017.
- [31] Vyperlang, “Vyper”, 2023. [Online]. Available: <https://github.com/vyperlang/vyper>. [Accessed: Dec. 15, 2023].
- [32] N. Grech, S. Lagouvardos, I. Tsatiris, and Y. Smaragdakis, “Elipmoc: Advanced decompilation of Ethereum smart contracts”, *Proceedings of the ACM on Programming Languages*, vol. 6, no. OOPSLA1, pp. 1–27, 2022.
- [33] J. Feist, G. Grieco, and A. Groce, “Slither: a static analysis framework for smart contracts”, in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019, pp. 8–15.
- [34] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, “SmartCheck: Static analysis of Ethereum smart contracts”, in *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*, 2018, pp. 9–16.
- [35] F. Ma, Z. Xu, M. Ren, Z. Yin, Y. Chen, L. Qiao, B. Gu, H. Li, Y. Jiang, and J. Sun, “Pluto: Exposing vulnerabilities in inter-contract scenarios”, *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4380–4396, 2021.
- [36] M. Mossberg, F. Manzano, E. Hennenfent, A. Groce, G. Grieco, J. Feist, T. Brunson, and A. Dinaburg, “Manticore: A user-friendly symbolic execution framework for binaries and smart contracts”, in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1186–1189.
- [37] Coinwire, “Beware of the LayerZero Token Scam: Stay Informed and Stay Safe”, 2022. [Online]. Available: <https://coinwire.com/beware-of-the-layerzero-scam-stay-informed-and-safe/>. [Accessed: Dec. 15, 2023].
- [38] Tangtj, “BSC Contract Database”, 2023. [Online]. Available: <https://github.com/tangtj/bsc-contract-database>. [Accessed: Dec. 15, 2023].
- [39] SvenMeyer, “Compiling 16 Files with 0.7.6 CompilerError”, 2023. [Online]. Available: <https://github.com/ethereum/solidity/issues/11638>. [Accessed: Dec. 15, 2023].
- [40] N. Lu, B. Wang, Y. Zhang, W. Shi, and C. Esposito, “NeuCheck: A more practical Ethereum smart contract security analysis tool”, *Software: Practice and Experience*, vol. 51, no. 10, pp. 2065–2084, 2021.
- [41] Forta, “Forta Network”, 2023. [Online]. Available: <https://forta.org/>. [Accessed: Dec. 15, 2023].
- [42] V. Buterin et al., “A next-generation smart contract and decentralized application platform”, *White Paper*, vol. 3, no. 37, pp. 2–1, 2014.
- [43] X. Tang, Z. Jia, and W. Yang, “Blockchain Application Status and Ecology”, in *Blockchain Application Guide: Methodology and Practice*, pp. 35–48, Springer, 2022.
- [44] R. Behnke, “EXPLAINED: THE QUBIT HACK (JANUARY 2022)”, 2022. [Online]. Available: <https://www.halborn.com/blog/post/explained-the-qubit-hack-january-2022>. [Accessed: Dec. 15, 2023].
- [45] R. Behnke, “EXPLAINED: THE METER.IO HACK (FEBRUARY 2022)”, 2022. [Online]. Available: <https://www.halborn.com/blog/post/explained-the-meter-io-hack-february-2022>. [Accessed: Dec. 15, 2023].

- [46] B. Guidi and A. Michienzi, “Sleepminting, the brand new frontier of Non Fungible Tokens fraud”, in Proceedings of the 2022 ACM Conference on Information Technology for Social Good, 2022, pp. 75--81.
- [47] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system”, 2008.
- [48] CoinMarketCap, “Bitcoin price today, BTC to USD live price, marketcap and chart | CoinMarketCap”, 2024. [Online]. Available: <https://coinmarketcap.com/currencies/bitcoin/>. [Accessed: Dec. 15, 2023].
- [49] Scam Sniffer, “Zero Transfer Scam”. [Online]. Available: <https://twitter.com/realScamSniffer/status/1755862295224459675>. [Accessed: Oct. 15, 2024].
- [50] Dune, “How Users React”. [Online]. Available: <https://dune.com/kimichi/how-users-react>. [Accessed: Oct. 15, 2024].
- [51] G. Ye et al., “Interface Illusions: Uncovering the Rise of Visual Scams in Cryptocurrency Wallets”, Proceedings of the ACM on Web Conference 2024, 2024.
- [52] N. Grech, L. Brent, B. Scholz, et al., “Gigahorse: Thorough, declarative decompilation of smart contracts”, in 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), IEEE, 2019, pp. 1176--1186.
- [53] Z. Liao, Y. Nan, H. Liang, et al., “Smartaxe: Detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis”, Proceedings of the ACM on Software Engineering, vol. 1, no. FSE, pp. 249--270, 2024.
- [54] K. Wang, Y. Li, C. Wang, et al., “XGuard: Detecting Inconsistency Behaviors of Crosschain Bridges”, in Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, 2024, pp. 612--616.
- [55] BscScan, “TroyEmpire”, 2024. [Online]. Available: <https://bscscan.com/address/0xb48200ed722e7e86a78d04245bf743d047289e95#code>. [Accessed: Jun. 7, 2024].
- [56] BscScan, “SecondLive”, 2024. [Online]. Available: <https://bscscan.com/address/0x260e69ab6665b9ef67b60674e265b5d21c88cb45#code>. [Accessed: Jun. 7, 2024].
- [57] BscScan, “TTGame”, 2024. [Online]. Available: <https://bscscan.com/address/0x7192611537108231ce07ac20ddf40c850a00ef6a#code>. [Accessed: Jun. 7, 2024].
- [58] BscScan, “WalletLocking”, 2024. [Online]. Available: <https://bscscan.com/address/0xb89cb9297c29ca0b4ae1d2f0b68089c9f39017ce#code>. [Accessed: Jun. 7, 2024].
- [59] T. Chen, Y. Zhang, Z. Li, et al., “TokenScope: Automatically detecting inconsistent behaviors of cryptocurrency tokens in Ethereum”, in Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, 2019, pp. 1503--1520.
- [60] Our Team, “Error in displaying msg.sender in Transaction Logs”, 2024. [Online]. Available: <https://github.com/blockscout/blockscout/issues/9879>. [Accessed: Jun. 7, 2024].
- [61] Ruaro, Nicola and Gritti, Fabio and McLaughlin, et al., “Not your Type! Detecting Storage Collision Vulnerabilities in Ethereum Smart Contracts”, in Netw. Distrib. Syst. Security Symp, 2024.
- [62] Gritti, Fabio and Ruaro, Nicola and McLaughlin, et al., “Confusum contractum: confused deputy vulnerabilities in ethereum smart contracts”, in 32nd USENIX Security Symposium (USENIX Security 23), pp. 1793--1810, 2023.

[63] Ghaleb, Asem and Rubin, Julia and Pattabiraman, et al., “eTainter: detecting gas-related vulnerabilities in smart contracts”, in Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 728–739, 2022.

附录 A. 真实世界攻击示例

A. 我们分类体系中五类攻击向量的示例

事件仿冒。 事件仿冒的两个知名实例是 Qubit Bridge [28] 和 Meter Bridge [45] 遭受的攻击，它们分别造成了 8,000 万美元和 430 万美元损失。

不一致日志。 许多此类问题是在 2024 年 4 月 1 日 BSC 链上 25 个最活跃的智能合约中发现的。具体而言，TroyEmpire [55]、SecondLive [56]、WalletLocking [58] 和 TTGameEvents [57] 的合约被发现存在这些问题。例如，TTGame 项目会为用户注册生成一个地址，并向该地址转入一定数量的 BNB，作为未来自动调用的 gas 费用。通过从历史交易中识别授权用户地址，我们可以发现其余发出幻影事件的未授权交易。尽管其他三个项目的用户权限并不明确，但我们可以模拟调用其函数并任意生成取款事件，从而确认这些问题的存在。

合约仿冒。 对于合约仿冒攻击，最值得注意的实例是 pNetwork 攻击，该攻击造成 1,270 万美元损失。这一点已在第 III-C 节中简要介绍。类似地，THORChain 因错误处理器问题遭受了 800 万美元损失 [8]。关于仿冒合约攻击，一个代表性案例是“sleepminting”攻击，该攻击因生成一个估值 6,900 万美元的 NFT 并将其上架平台而受到关注。该事件已在学术界被广泛研究 [10]、[15]、[46]。根据 Forta 平台 [41] 的最新信息，从 2023 年 12 月 3 日至 12 月 9 日的短短 7 天内，就生成了 8,130 条 sleepminting 告警。

转账事件欺骗。 我们对链上数据的分析揭示了大量转账事件欺骗实例，其中 ERC-20 代币和 NFT 事件被篡改了发送者地址。这些地址包括类似名人、知名交易所和其他引人注意实体的伪造地址。若干此类案例已导致重大财务损失 [37]、[9]、[22]、[49]、[51]。

事件处理错误。 一个著名的事件处理错误示例涉及 Rarible 和 OpenSea 因 sleepminting 而经历的错误信息案例。Etherscan 和 Bscscan 等区块链浏览器以及各种钱包，通常会将伪造转账事件视为合法交易。另一个实例涉及 blockwell.ai，其中钱包错误地将来自欺骗代币的幻影事件识别为有效 ERC-20 转账 [17]。类似漏洞也出现在 ERC-721 和 ERC-1155 代币中。

此外，最大的 NFT 市场 OpenSea 也面临类似问题。攻击者操纵 OpenSea 展示的 NFT 元数据，嵌入恶意载荷，触发非预期钱包行为，最终使攻击者获利。类似攻击也已在 ERC-20 项目中发现，这表明在整个区块链生态系统中安全处理事件数据存在广泛困难 [14]、[13]。

B. 不一致日志和合约仿冒的框架

图 8：不一致日志应用概览。

图 8 给出了应用中不一致日志的概览。在该框架中，用户通过应用进行操作，应用与数据库交互以插入日志，并与区块链上的智能合约交互以发出事件。然而，由于日志记录实践存在差异，数据库日志与区块链交易日志之间可能出现不一致。攻击者可以通过直接调用记录函数来利用这一点，进而可能造成应用内部日志记录与区块链交易日志之间的不匹配。

图 9：两类合约仿冒。

(a) 混合事件攻击； (b) 仿冒合约攻击。

图 9 展示了两类合约仿冒攻击。在混合事件攻击中，恶意合约与真实合约交互，来自两个合约的日志被记录在同一交易中，这可能误导验证者。在仿冒合约攻击中，攻击者部署一个模仿真实合约的合约，生成看似源自真实合约的欺骗性日志，从而进一步混淆验证者。

C. 转账事件欺骗案例

图 10：空投事件欺骗。

图 10 展示了一个转账事件欺骗攻击示例，其中攻击者伪造事件发送者，导致区块链浏览器和钱包错误地将其显示为成功转账。这种误导性表示可以欺骗用户，并被作为社会工程策略来利用用户对这些界面的信任。

表 5：转账事件欺骗攻击交易的真实案例。

	Sender	Transfer From	Transfer To	Token
1	Victim	Victim	0x734659...50Ca79F7	USDT
2	Attacker	Victim	0x73435A...2Bca79F7	FAKE USDT
3	Victim	Victim	0x73435A...2Bca79F7	USDT

2024 年 2 月 9 日的转账欺骗攻击中，攻击者成功执行了一次转账事件欺骗攻击，造成 104 万美元损失。该攻击利用了钱包和区块链浏览器在解析和显示转账事件时存在的漏洞。如表 5 所示，在用户向合法地址（0x734...a79F7）发送资金后，攻击者伪造了一个转账事件，使其看起来像是资金被发送到了一个由攻击者控制的相似地址。由于某些钱包和浏览器处理这些事件的方式，虚假转账被显示为合法转账，导致用户误向攻击者地址发送额外资金，造成重大财务损失 [49]。

术语说明

- **blockchain**: 区块链。
- **Ethereum**: 以太坊。
- **Ethereum Virtual Machine / EVM**: 以太坊虚拟机 / EVM。
- **smart contract**: 智能合约。
- **transaction**: 交易。
- **transaction log**: 交易日志。本文中特指交易执行期间生成并存储在链上的日志记录，不与“事件日志”混用。
- **event log**: 事件日志。通常指围绕事件语义进行查询和监听的日志记录。
- **event**: 事件。
- **log forgery**: 日志伪造。
- **forged log**: 伪造日志。
- **phantom event**: 幻影事件。
- **legitimate event**: 合法事件。
- **authentic event**: 真实事件。原文在安全模型中使用 **authentic events**，为避免与用户指定的 **legitimate event** 混淆，译为“真实事件”。
- **token transfer**: 代币转账。
- **contract address**: 合约地址。
- **externally owned account / EOA**: 外部拥有账户 / EOA。
- **decentralized application / DApp**: 去中心化应用 / DApp。
- **decentralized exchange / DEX**: 去中心化交易所 / DEX。
- **log emitter / emitter**: 日志发出者 / 发出者。本文中 $S_{\{address\}}$ 表示发出事件的智能合约，不等同于交易发送者。
- **caller**: 调用者。
- **callee**: 被调用者。
- **trace**: 执行轨迹。
- **transaction receipt**: 交易回执。
- **topic**: 主题。
- **data field**: 数据字段。
- **vulnerability**: 漏洞。
- **attack**: 攻击。
- **exploit**: 漏洞利用。
- **detection**: 检测。
- **dataset**: 数据集。
- **empirical study**: 实证研究。
- **false positive**: 误报。
- **false negative**: 漏报。
- **ground truth**: 真实标签 / 基准事实。
- **Event Counterfeiting**: 事件仿冒。指合法合约中不同函数或路径可发出相同事件，从而使攻击者构造看似合法的幻影事件。
- **Inconsistent Logging**: 不一致日志。指链上事件发出与链下数据库日志或合约状态之间发生不一致。

- **Contract Imitation:** 合约仿冒。指通过恶意合约模仿或混合真实合约事件来误导链下系统。
- **Transfer Event Spoofing:** 转账事件欺骗。指通过伪造转账事件进行用户误导或社会工程攻击。
- **Event Handling Error:** 事件处理错误。指链下应用在解析、展示、插入或存储事件时产生错误。
- **Blended Event Attack:** 混合事件攻击。
- **Mimicry Contract Attack:** 仿冒合约攻击。
- **off-chain relay:** 链下中继器。
- **bytecode:** 字节码。
- **source code:** 源代码。
- **intermediate representation / IR:** 中间表示 / IR。
- **inter-contract control flow graph / ICFG:** 跨合约控制流图 / ICFG。
- **control flow graph / CFG:** 控制流图 / CFG。
- **backward taint analysis:** 后向污点分析。
- **interprocedural backward taint tracking:** 跨过程后向污点跟踪。
- **symbolic execution:** 符号执行。
- **SMT solver:** SMT 求解器。
- **precision:** Precision, 本文实验表中保留英文指标名; 可理解为精确率。
- **recall:** Recall, 本文实验表中保留英文指标名; 可理解为召回率。
- **F1:** F1 得分。

翻译注意点

- 原 PDF 为双栏版式，直接提取文本会出现列间交错；本译稿依据 arXiv 源文件中的 LaTeX 内容翻译，并按原论文逻辑顺序整理。
- 原文中存在少量拼写或语法问题，例如 `Cross-Chain Brdiges`、`Fesit`、`data set` 等。本译稿在中文中按语义译出，但保留工具名、合约名、函数名、字段名和数学符号原样。
- `event`、`event log` 与 `transaction log` 在本文中关系紧密但语义不同：事件是智能合约发出的语义对象；交易日志是事件在链上记录的日志结构；事件日志用于强调围绕事件进行监听和查询的日志。译文已按上下文区分。
- `emitter`、`TX_sender` 与 `TokenSender` 不能混同：`emitter` 是发出事件的合约；`TX_sender` 是创建交易的用户地址；`TokenSender` 是事件中记录的代币发送者。
- 原文 `authentic events/contracts` 在安全模型中与 `forged contracts` 相对，译为“真实事件/真实合约”；用户指定术语中的 `legitimate event` 仍译为“合法事件”。
- 原文 `Event Counterfeiting` 直译可为“事件伪造”，但为避免与总主题 `event forgery/log forgery` 混淆，本文统一译为“事件仿冒”，并在术语说明中标注其含义。
- 原文参考文献采用 LaTeX citation key；译稿已根据 `bibliography.tex` 中的出现顺序转换为数字引用，例如 `bitcoin` 对应 [47]、`coinmarketcap` 对应 [48]、`zero_transfer` 对应 [49]。
- 原文 `Acknowledgments` 文件中全部为注释和 TBD，论文 PDF 未呈现正式致谢正文；译稿中注明该部分为占位内容。
- 原文附录表 5 中有颜色高亮地址前后缀的 LaTeX 标记，译稿保留地址文本本身，不保留颜色标记。